

Análise de Crimes em Chicago

Pipeline Big Data com Apache Spark

7,9M
registros

1,9 GB
dados brutos

23
colunas

3
modelos ML

Apache Spark 3.5

PySpark MLlib

Docker Compose

Parquet

Classificação binária — prever se um crime resultará em prisão

Objetivo e Escolha do Problema

O que queremos prever?

Dado um registro de crime em Chicago, prever se haverá **prisão** ou não — variável **Arrest**: 0 (sem prisão) ou 1 (com prisão).

É um problema de **classificação binária supervisionada** com dados reais de segurança pública de 2001 a 2017.

Por que esse dataset?

- Cobre **16 anos** de registros policiais reais de Chicago
- **1,9 GB** de dados — atende o requisito de Big Data
- Rica em features: tipo de crime, local, horário, distrito, coordenadas
- Desbalanceamento de classes realista — exige tratamento explícito
- Disponível publicamente no **Kaggle**

A pergunta "esse crime vai resultar em prisão?" tem valor prático: entender quais fatores mais influenciam o resultado ajuda a identificar padrões de policiamento e efetividade por tipo de crime e região.

Parte 1 — Ambiente de Big Data com Docker

Arquitetura do cluster

Container	Função	Recursos
spark-master	Coordena o cluster	—
spark-worker-1	Processa partições	2 cores · 2 GB
spark-worker-2	Processa partições	2 cores · 2 GB
jupyter	Driver PySpark	2 GB

Monitoramento em tempo real via Spark UI em `localhost:4040`

Por que Docker e por que Spark?

- **Docker** garante reprodutibilidade total — mesmo ambiente em qualquer máquina, sem conflito de dependências
- **Docker Compose** orquestra os 4 containers com um único comando
- **Apache Spark** processa dados em memória de forma distribuída — escala com o volume sem reescrever o código
- Os 2 workers processam partições do dataset **em paralelo**, reduzindo o tempo de execução

O cluster simula um ambiente de produção real — o mesmo código rodaria em AWS EMR ou Databricks sem modificações.

O Dataset — Chicago Crimes (Kaggle)

4 arquivos CSV — total 1,9 GB

Período	Tamanho
2001–2004	453 MB
2005–2007	449 MB
2008–2011	646 MB
2012–2017	350 MB

O que cada linha representa

Um registro policial: tipo do crime, data e hora, endereço, distrito, se houve prisão, coordenadas GPS e descrição do local.

Colunas utilizadas no modelo

Coluna	Tipo	Papel
Primary Type	Categórica	Feature
Location Description	Categórica	Feature
District / Beat	Numérica	Feature
Latitude / Longitude	Numérica	Feature
Domestic	Booleana	Feature
Hour / DayOfWeek / Month	Numérica	Feature (derivada)
Arrest	Booleana	Target (0/1)

Parte 1 — Ingestão e Conversão para Parquet

4 CSVs (1,9 GB)



Leitura paralela no Spark



DataFrame unificado



Gravação Parquet



551 MB (comprimido)

Por que converter para Parquet?

- **Formato colunar** — lê somente as colunas solicitadas, ignorando o restante. CSV lê tudo, sempre
- **Compressão Snappy integrada** — 1,9 GB → 551 MB sem perda de dados (70% menor)
- **Schema embutido** — os tipos já estão salvos, sem necessidade de re-inferência
- **Leitura 5-10× mais rápida** nas etapas de ETL e ML seguintes

Por que isso importa neste projeto?

O pipeline tem **4 fases** que leem os dados. Converter para Parquet uma vez e reler nas fases seguintes economizou dezenas de minutos de I/O.

Com CSV, cada fase releria 1,9 GB do disco. Com Parquet, lê apenas as colunas necessárias em 551 MB já comprimidos.

Decisão de engenharia padrão em pipelines Big Data de produção.

Parte 2 — Pré-processamento: Limpeza dos Dados

Etapa	O que foi feito	Por que foi necessário	Registros restantes
Leitura do Parquet	7.941.286 registros carregados	Ponto de partida	7.941.286
Remoção de nulos	dropna() nas colunas críticas	Linhas sem tipo de crime, data ou resultado de prisão são inutilizáveis para qualquer análise ou modelo	7.834.133
Filtro de coordenadas	Remove lat/lon = 0	Coordenada (0,0) é o oceano Atlântico — indica ausência de dado geográfico, não localização real	—
Seleção de colunas	Mantém 10 colunas relevantes	ID, Case Number, IUCR e FBI Code não têm poder preditivo e aumentam custo de processamento	—
Dropna final	Remove nulos restantes nas features	O modelo de ML exige que todas as features de entrada estejam presentes	7.145.306

796.000 registros removidos (~10%) — principalmente por ausência de coordenadas geográficas válidas. Essa perda é aceitável: os registros descartados não carregam informação utilizável.

Parte 2 — Feature Engineering

Features derivadas do timestamp

Feature criada	De onde vem	Por que importa
Hour	Hora da ocorrência	Prisões têm padrão circadiano claro — pico às 11h e 19–21h
DayOfWeek	Dia da semana	Fins de semana têm perfil de crimes diferente de dias úteis
Month	Mês do ano	Captura sazonalidade — alguns crimes concentram em épocas específicas

O timestamp bruto não pode entrar no modelo — é texto. Extrair componentes numéricos transforma informação temporal em features utilizáveis.

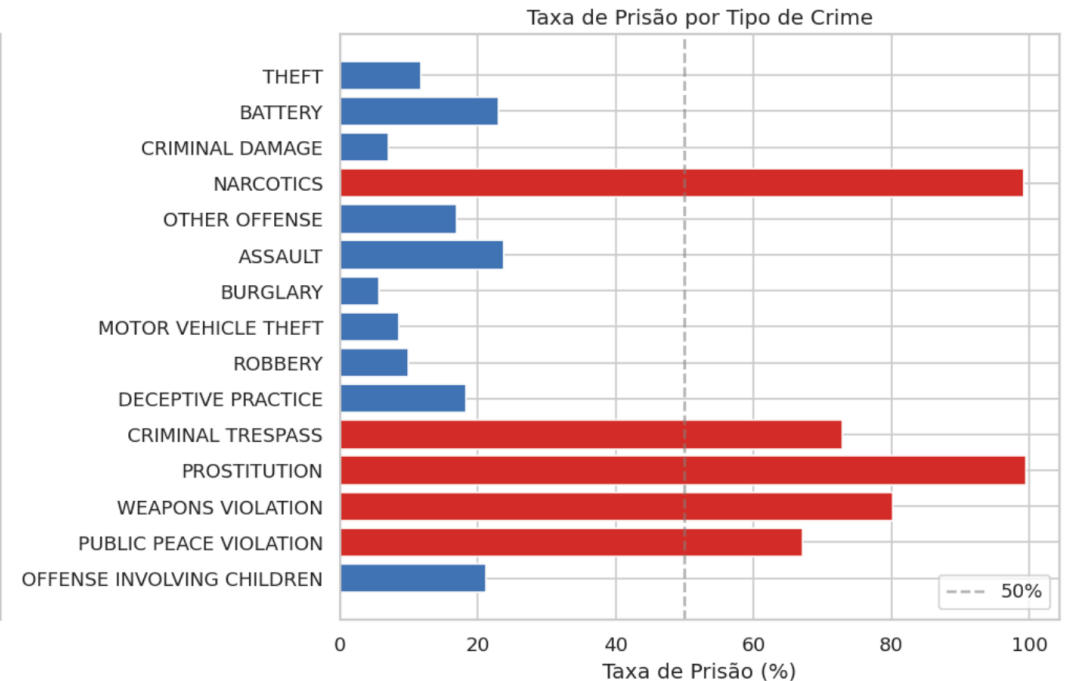
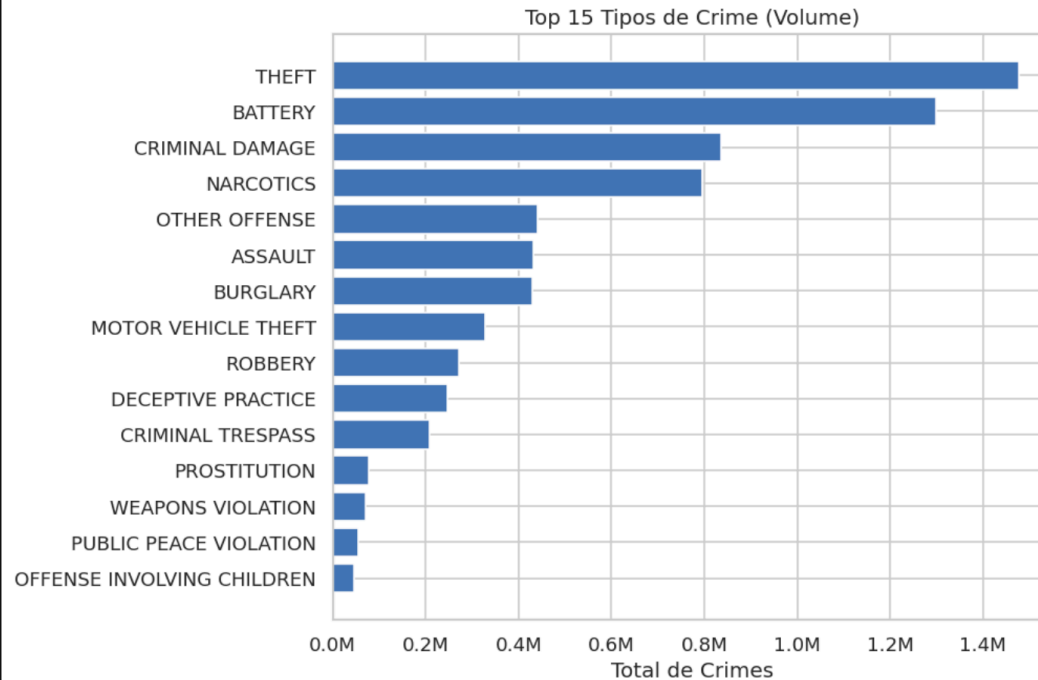
Outras transformações

- **Arrest_label** — converte "True"/"False" (string) para 1/0 (inteiro). O modelo exige numérico
- **Domestic_int** — mesma lógica para a coluna de violência doméstica
- **District, Beat, Community Area** — convertidos de string para inteiro
- **Latitude e Longitude** — convertidos de string para double para entrar no VectorAssembler

Essas conversões são obrigatórias — o Spark MLlib não aceita tipos mistos no vetor de features.

EDA — Distribuição por Tipo de Crime

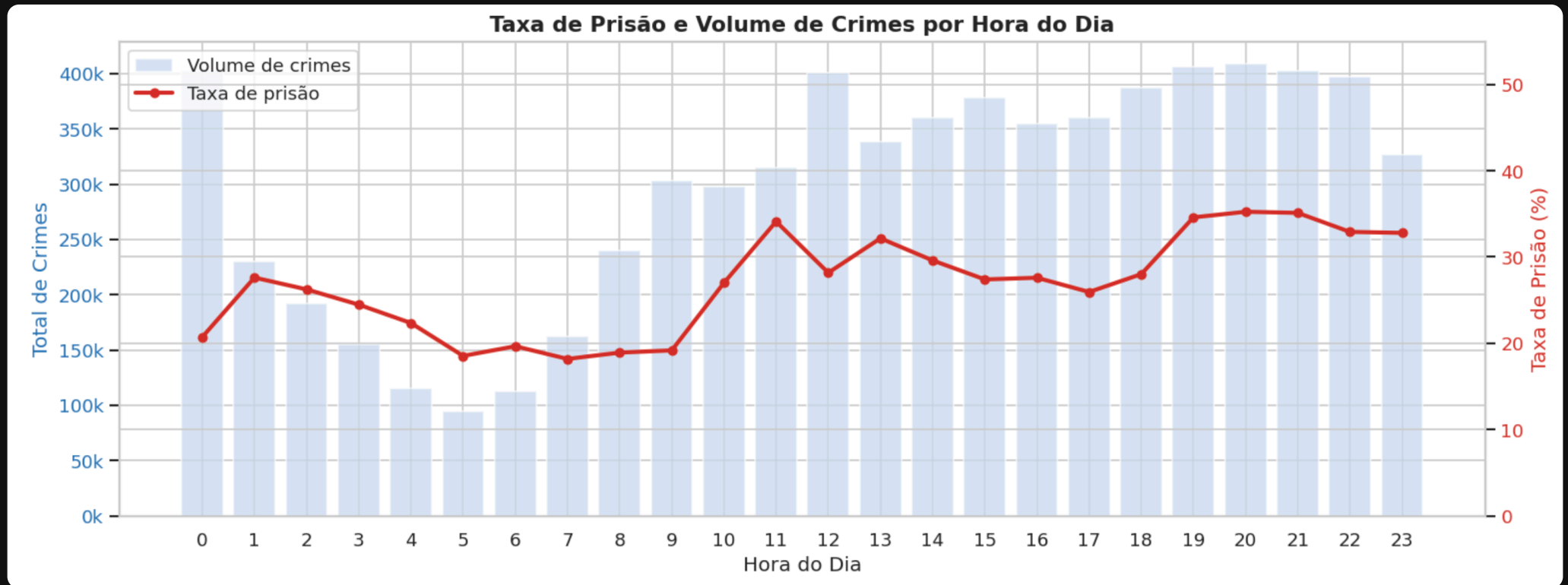
Distribuição de Crimes por Tipo — Chicago



Theft e Battery lideram em volume — juntos representam mais de 35% de todos os crimes, mas com taxa de prisão baixa (12% e 23%), pois geralmente ocorrem sem testemunhas imediatas.

NARCOTICS: 99,21% de taxa de prisão — maior outlier do dataset. Tráfico é flagrante por natureza. Esse padrão distorcerá os modelos de ML.

EDA — Taxa de Prisão por Hora do Dia

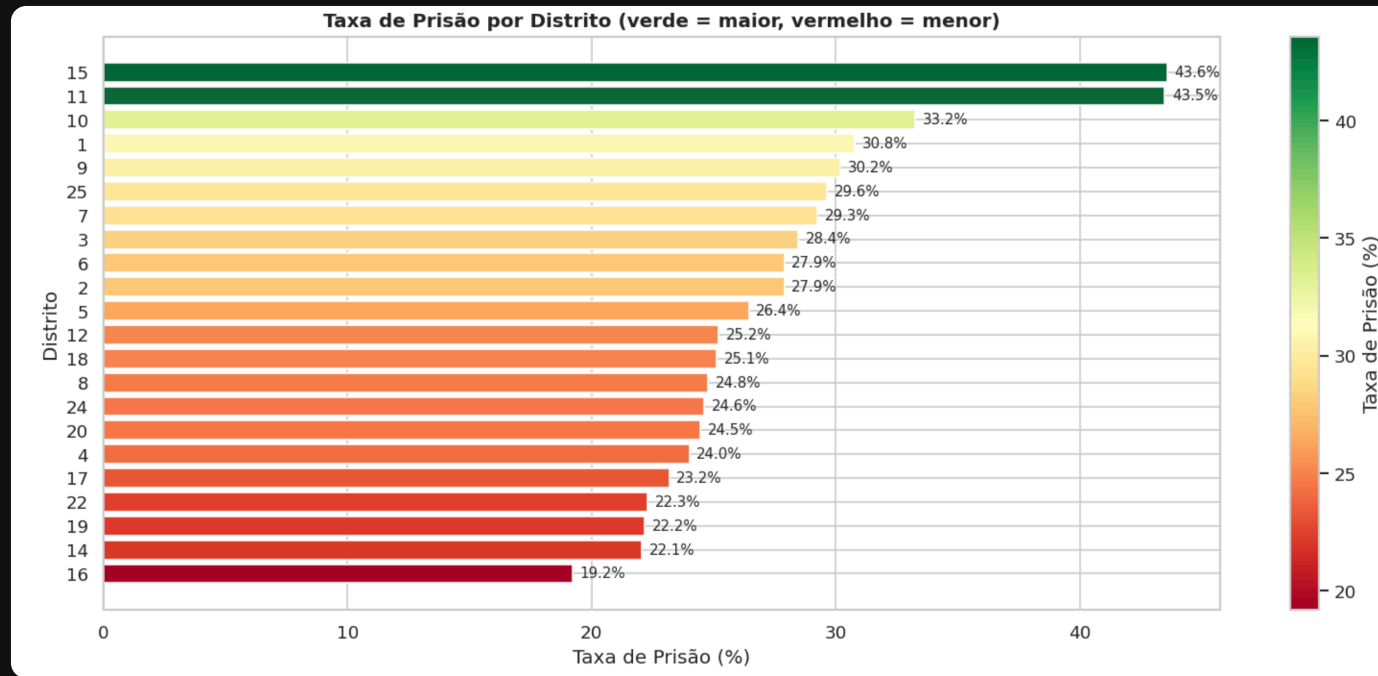


Barras azuis: volume total de crimes · Linha vermelha: taxa de prisão (%)

Pico de prisões às 11h (~34%) — não é o horário de maior volume de crimes. Indica *ação policial proativa*: operações planejadas no período da manhã, não reação a crimes espontâneos.

Menor taxa na madrugada (5h-8h: ~18%) — crimes ocorrem, mas menor presença policial reduz a chance de flagrante.

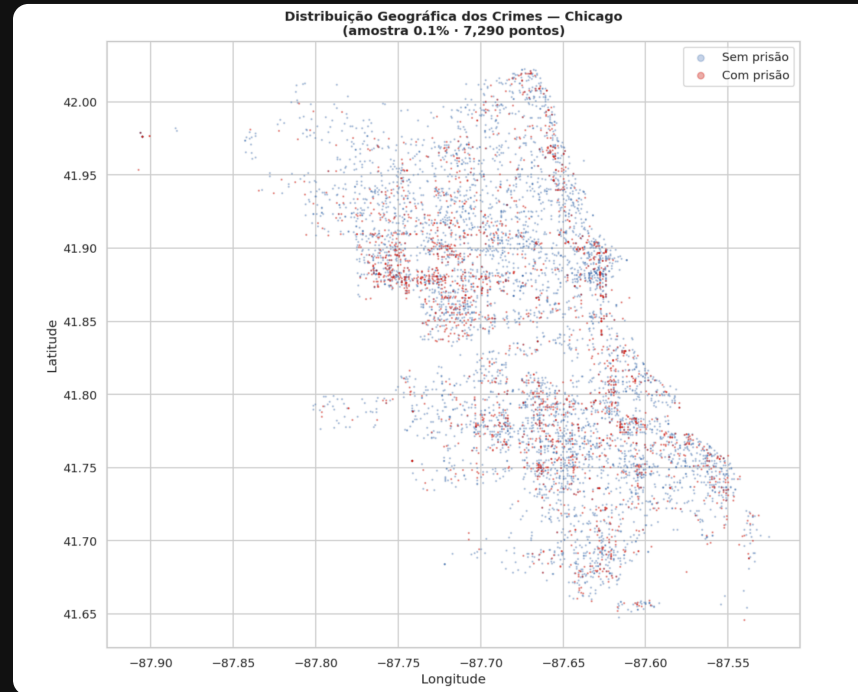
EDA — Taxa de Prisão por Distrito



Distritos 15 e 11 (43,6% e 43,5%) — maior efetividade de prisão. Concentram maior proporção de crimes de NARCOTICS (99% de prisão garantida).

Distrito 16 (19,2%) — menor taxa. Diferença de 2,3× em relação ao Distrito 15 revela desigualdade de efetividade policial entre regiões.

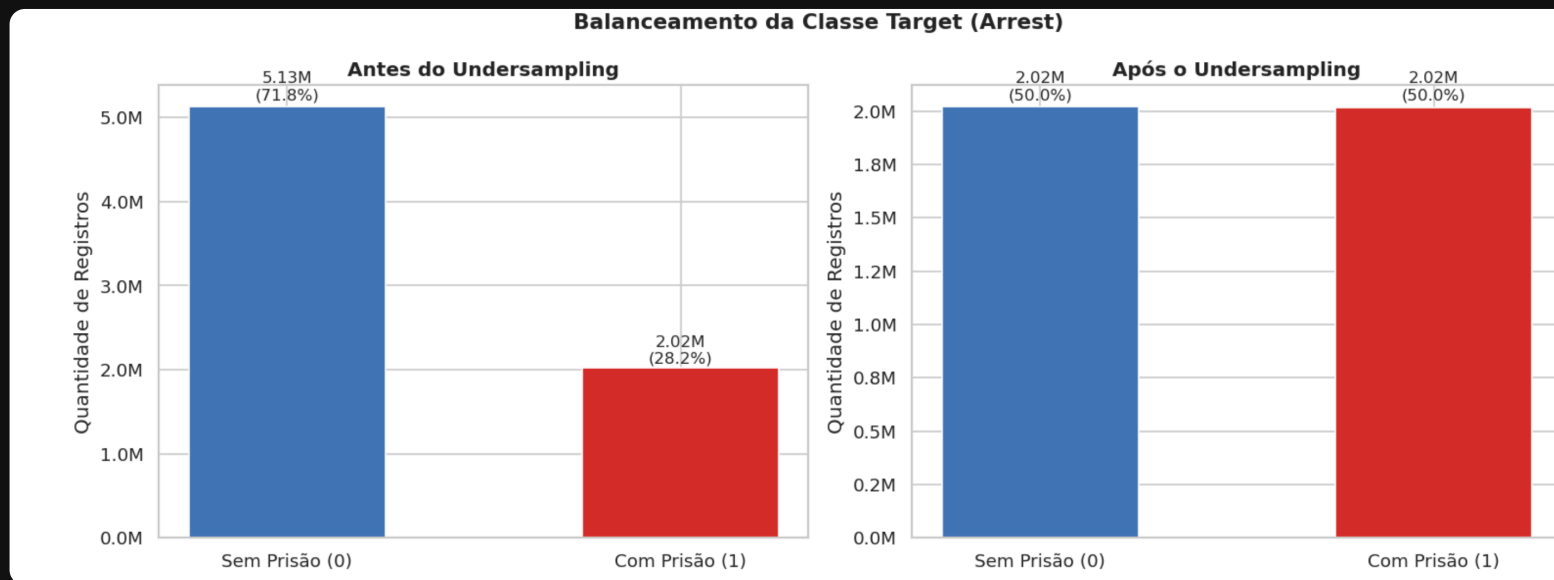
EDA — Distribuição Geográfica dos Crimes



Amostra de 0,1% dos registros (~7.290 pontos) · Azul: sem prisão · Vermelho: com prisão

A concentração de crimes é maior no centro-sul de Chicago. Crimes com e sem prisão seguem a mesma distribuição espacial — **não há separação geográfica clara entre as classes**. A localização sozinha não é um bom preditor, mas contribui em combinação com outras features.

Pré-processamento — Balanceamento de Classes



Por que o desbalanceamento é um problema?

Com 71,8% de casos "sem prisão", um modelo que *sempre prevê zero* acertaria 71,8% sem aprender absolutamente nada. As métricas ficariam boas sem nenhuma inteligência real.

Solução: Undersampling

Reduzimos a classe majoritária (5,1M) para igualar a minoritária (2,0M), resultando em **4,0M de registros balanceados (50/50%)**.

Preferimos undersampling a dados sintéticos (SMOTE) — mantemos apenas registros reais, sem risco de introduzir ruído artificial.

Parte 3 — Pipeline de Machine Learning



StringIndexer

Converte categorias de texto em índices numéricos ordenados por frequência.

Necessário porque algoritmos de ML trabalham apenas com números — não entendem texto diretamente.

OneHotEncoder

Transforma cada índice em vetor binário esparsos, eliminando a falsa ordem numérica.

Sem isso, o modelo entenderia "THEFT=0 é menor que BATTERY=1" — o que não faz sentido.

VectorAssembler

Concatena todas as features em um único vetor de **206 dimensões**.

Split: 80% treino (3,23M linhas) · 20% teste (807K linhas) · seed fixo para reprodutibilidade.

O pipeline é treinado (*fit*) apenas no conjunto de treino e aplicado (*transform*) no teste — sem data leakage.

Os 3 Modelos Escolhidos

Logistic Regression

Baseline

Modelo linear clássico. Estabelece a referência mínima — qualquer modelo mais complexo deve superar esse resultado.

Interpretável e escalável: o Spark distribui o gradiente entre os workers sem carregar tudo na memória de uma vez.

Decision Tree

Interpretável

Captura relações não-lineares sem necessidade de normalização. A principal vantagem é a **feature importance** — revela quais variáveis mais influenciam a decisão.

Profundidade máxima 5 para evitar overfitting.

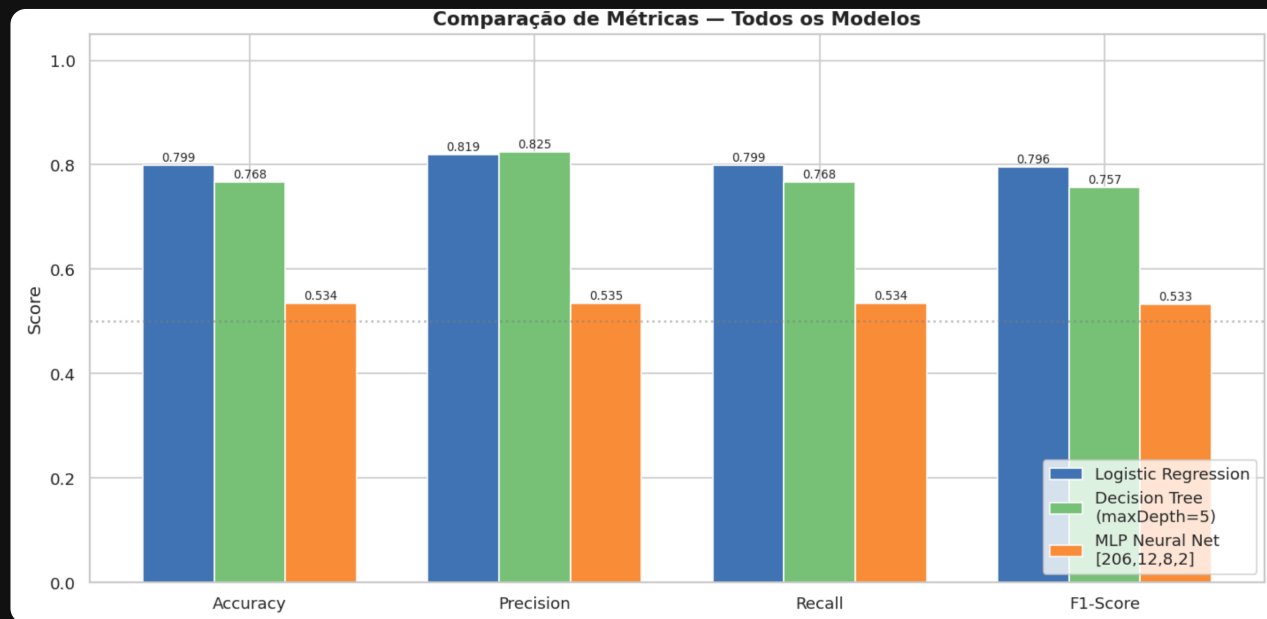
MLP Neural Net

Rede Neural

Arquitetura com 4 camadas: 206 → 12 → 8 → 2. Capaz de aprender padrões não-lineares complexos.

Limitação: o otimizador L-BFGS carrega o dataset inteiro por iteração — inviável com 3,2M linhas em 4 GB de RAM.

Resultados — Comparação dos Modelos



Logistic Regression

79,9%

Accuracy · F1: 0,796

Melhor modelo geral — equilibrado em todos os erros

Decision Tree

76,8%

Accuracy · F1: 0,757

Melhor Precision (82,5%) — poucos falsos positivos

MLP Neural Net

53,4%

Accuracy · F1: 0,533

Próximo do acaso (50%) — treinado em apenas 10% dos dados

Análise de Erros — Confusion Matrix

Confusion Matrix — Comparação dos Modelos

Logistic Regression			
Valor Previsto	Previsto: 0	VN 373,529 (46.3%)	FN 131,594 (16.3%)
	Previsto: 1	FP 30,484 (3.8%)	VP 271,461 (33.6%)
		Verdadeiro: 0	Verdadeiro: 1
		Valor Verdadeiro	

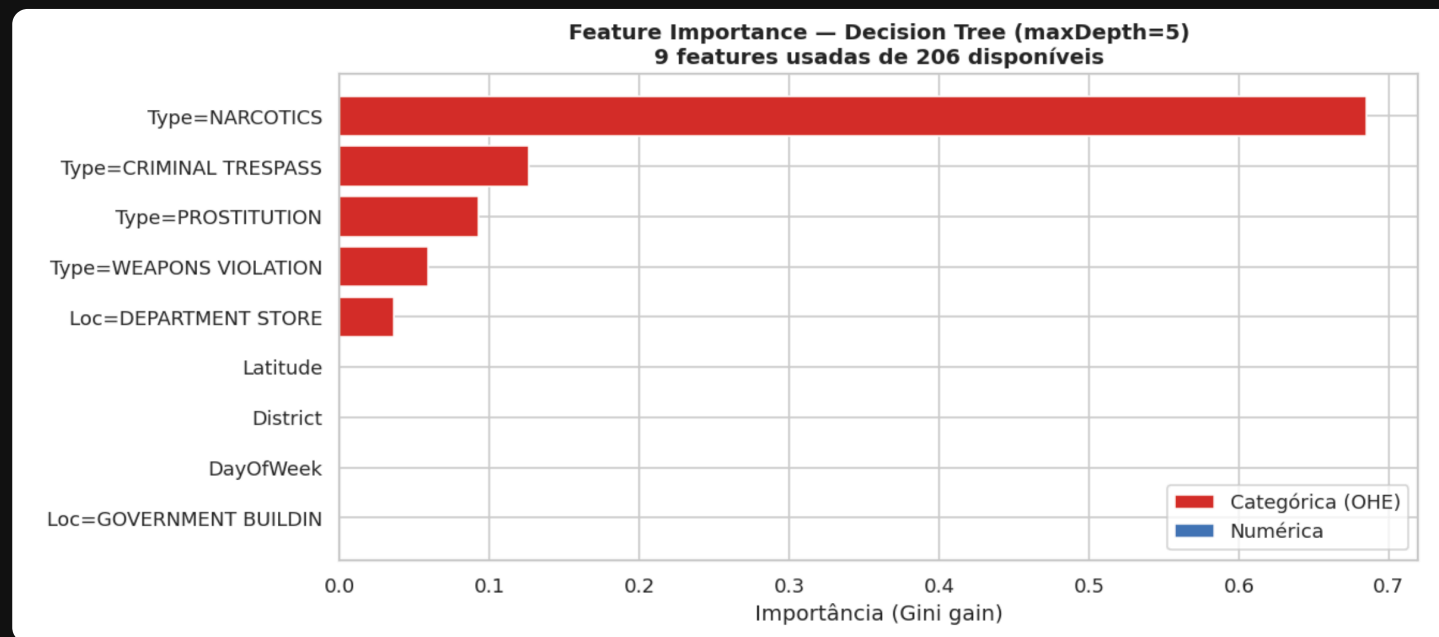
Decision Tree		
Valor Previsto		
Previsto: 0	VN 395,202 (49.0%)	
Previsto: 1	FP 8,811 (1.1%)	
	Verdadeiro: 0	Verdadeiro: 1
	Valor Verdadeiro	
	FN 178,692 (22.1%)	VP 224,363 (27.8%)

MLP Neural Network		
Valor Previsto	Previsto: 0	Previsto: 1
	VN 198,376 (24.6%)	FN 170,201 (21.1%)
Valor Verdadeiro	Verdadeiro: 0	Verdadeiro: 1
	FP 205,637 (25.5%)	VP 232,854 (28.9%)

LR vs DT — tipos de erro diferentes: a LR erra mais em falsos positivos (3,8%) mas captura bem as prisões reais. A DT é mais conservadora — apenas 1,1% de FP, mas perde 22,1% das prisões reais (falsos negativos).

MLP — distribuição uniforme (~25% em cada quadrante): o modelo não aprendeu padrão algum. Com apenas 10% dos dados e 20 iterações, ficou essencialmente aleatório.

O que o Modelo Aprendeu — Feature Importance



NARCOTICS domina com 0,69 de Gini gain — 69% da capacidade decisória da árvore está em uma única feature. A árvore aprendeu: "se for NARCOTICS, prende". Isso reflete o dado real (99,21% de prisão).

9 de 206 features foram usadas — com profundidade máxima 5, a árvore não tem espaço para aproveitar features numéricas que exigem splits mais refinados.

Análise Crítica dos Resultados

Por que a Logistic Regression superou a Decision Tree?

- A Decision Tree com profundidade 5 cria apenas 31 folhas — com 206 features, isso é muito restritivo
- A árvore usou apenas 9 features; a LR considera todas as 206
- Um modelo simples venceu um complexo porque o complexo estava mal configurado
- Com profundidade 10–15 a DT provavelmente inverteria o resultado

Por que o MLP ficou em 53%?

- Treinado em apenas **10% dos dados** por limitação de memória
- O otimizador L-BFGS carrega o dataset inteiro por iteração
- Redes neurais precisam de muito mais dados e épocas para convergir

O viés do NARCOTICS nas métricas

NARCOTICS representa **10% do volume** mas tem **99,21% de taxa de prisão**. Cria um atalho trivial:

- A Decision Tree resolve ~800K registros com um único split no primeiro nível
- A LR aprende um peso altíssimo para essa categoria
- As métricas gerais (79,9%) são parcialmente infladas por essa facilidade
- Uma avaliação mais rigorosa analisaria separadamente com e sem NARCOTICS

Os modelos são bons — mas parte do desempenho vem de um padrão trivialmente fácil no dataset, não de inteligência real.

Desafios Enfrentados

SparkContext morreu por falta de memória

Durante o treino do MLP, os 4 GB de RAM do cluster se esgotaram e o SparkContext foi encerrado silenciosamente. Execuções seguintes falhavam com *AssertionError: sc is not None*.

Solução: recovery guard — o código verifica se o Spark está vivo antes do MLP e reinicia automaticamente, recarregando os dados do checkpoint em Parquet.

Encoding inconsistente quebrou o MLP

O pipeline treinado na amostra de 10% aprendeu 170 categorias ao invés de 206. Ao transformar o teste completo: *ArrayIndexOutOfBoundsException: index 206 out of bounds for double[170]*.

Solução: separar encoding do treino. O pipeline de features é ajustado no dataset completo (206 features). Apenas o classificador MLP treina na amostra.

MLP travava indefinidamente

Com 3,2M linhas e 206 features, o L-BFGS tentava carregar o dataset inteiro por iteração. A célula ficava com [*] sem progredir por horas.

Solução: amostrar 10% do treino (~323K linhas) e reduzir de 50 para 20 iterações.

Tempo de execução elevado

Cada execução completa levou **20–40 minutos**. Foram necessárias 5 execuções para depurar e validar o pipeline.

Estratégia: checkpoint do dataset balanceado em Parquet. Em caso de falha, reiniciamos apenas a partir da Parte 3 — sem reprocessar os 7,9M de registros.

Limitações Honestas

Do dataset

- **Dados históricos (2001–2017)** — padrões de policiamento e legislação mudaram
- **Crimes não reportados são invisíveis** — o dataset representa o que foi registrado, não o que aconteceu de fato. Viés de seleção inevitável
- **107.000 registros sem coordenadas válidas** foram descartados
- **NARCOTICS com 99% de prisão** distorce todas as métricas gerais
- **Sem dados socioeconômicos** por bairro — renda e densidade teriam poder preditivo real

Dos modelos e do hardware

- **MLP com 10% dos dados** — o resultado de 53% não é comparável com LR e DT em condições iguais
- **Decision Tree com profundidade 5** — subutiliza as 206 features disponíveis
- **Sem cross-validation (k-fold)** — avaliação com split único tem variância alta
- **Sem tuning de hiperparâmetros** — nenhum GridSearch por custo computacional
- **Cluster local com 4 GB de RAM total** — impede treino do MLP completo e uso de modelos ensemble

O que Gostaríamos de ter Feito

Modelos não implementados

- **Random Forest e Gradient Boosted Trees** — modelos ensemble que quase sempre superam uma árvore isolada. Descartados por uso intensivo de memória
- **MLP no dataset completo** — com cluster maior ou mini-batch SGD, treinaria nos 3,2M registros
- **Cross-validation com k=5** — estimativas muito mais confiáveis, mas multiplica o tempo por 5

Análises não realizadas

- **Avaliação excluindo NARCOTICS** — ver como os modelos se saem sem o atalho fácil
- **Curvas ROC e AUC** — análise mais granular por threshold de decisão
- **Métricas separadas por tipo de crime**

Por que não fizemos?

Cada decisão foi **consciente**, não por falta de conhecimento:

- **Hardware:** 4 GB de RAM total é o principal limitante. Random Forest esgotaria a memória
- **Tempo:** cada execução leva 20–40 minutos. Com 5 execuções para depuração, não havia margem para experimentos adicionais
- **Escopo:** o foco foi construir um pipeline completo e funcional de ponta a ponta

Em AWS EMR ou Databricks, todos esses pontos seriam viáveis com o mesmo código — apenas com mais recursos de hardware.

Conclusões

Resultados finais

Modelo	Accuracy	Precision	F1-Score
Logistic Regression	79,9%	81,9%	0,796
Decision Tree (d=5)	76,8%	82,5%	0,757
MLP [206,12,8,2]	53,4%	53,5%	0,533

Treino: 3,23M linhas · Teste: 807K linhas · 206 features

Pipeline funcional de ponta a ponta: 1,9 GB de CSV → ETL → EDA
→ 3 modelos treinados e avaliados com 7,9M de registros em
cluster Spark distribuído.

O que aprendemos

- **Formato importa:** Parquet reduziu 1,9 GB para 551 MB e acelerou leituras em 5–10×
- **Balanceamento é não-negociável:** sem undersampling, qualquer modelo aprenderia a dizer "sem prisão" e acertaria 71,8% sem inteligência alguma
- **Hardware limita algoritmo:** a escolha do modelo foi parcialmente determinada pela RAM, não só pelo desempenho teórico
- **Modelos simples bem alimentados vencem redes neurais mal alimentadas:** LR com 206 features superou MLP treinado em 10% dos dados
- **Feature importance é tão valiosa quanto accuracy:** revelou que o modelo resolve 10% do dataset com um split trivial

Obrigado

7,9M

linhas processadas

1,9 GB

dados brutos

79,9%

melhor accuracy

206

features no vetor ML

Apache Spark 3.5

PySpark MLlib

Docker Compose

Parquet + Snappy

Logistic Regression

Decision Tree

MLP Neural Net